TIR

SOTA: How to explain a classifier?

Mastrorillo Noé, Tortosa Matthias, Pigeon Axel, Sinatti Antoine

Abstract

Abstract here

Contents

0.1 Introduction . . . 1

1 Explanation methods based on Shapley values **1**
1.1 Game theory preliminaries . . . 1
1.2 Shapley values for feature explanation . . . 2
1.3 Approximation . . . 3
1.4 Different methods . . . 3
1.5 Synthesis . . . 4

2 Perturbation model-agnostic methods **4**
2.1 LIME (Local Interpretable Model-Agnostic Explainer) . . . 5
2.2 Anchors . . . 6
2.3 Related . . . 7

3 Comparison of all methods **8**

4 Conclusion **8**

0.1 Introduction

1 Explanation methods based on Shapley values

Some of the most widely used methods for local features importance explanation are based on the Shapley values from game theory. These methods are so popular because they give a contribution score for each feature in the decision that is easily understood by the user. This contribution score has a sign and a magnitude, so that a positive score means that the features contributes positively to the decision and the opposite for a negative score. This result in straightforward visualizations of local features importance, as displayed in 1 that shows Shapley-based contribution scores of features for a prediction of a naive Bayes classification model on an instance of the famous Titanic dataset. The goal here is to predict if the person represented by x would have been able to survive from the Titanic. The model predicted the class *yes*, and the scores tell us that the main contribuing feature is the *Sex*: because the person is a female, she has good chances to survive. We can also see that the *Class* of the person works slightly against its survival, as well as the fact that its an *adult*.

While these methods are amongst the most popular, that doesn't mean that they are superior, as we will see the several shortcomings later.

1.1 Game theory preliminaries

In game theory, Shapley values provide a mathematically fair and unique method to attribute the payoff of a cooperative game to the players. Formally, a coalitional game $(\mathcal{F}, v)$ is composed of a set of n players $\mathcal{F} = \{1, \ldots, n\}$, and a characteristic function $v : \mathcal{P}(\mathcal{F}) \to \mathbb{R}$ that represents the payoff for a subset of players $\mathcal{S} \subset \mathcal{F}$ (a coalition). Then, the Shapley value for a single player $i \in \mathcal{F}$, noted $\varphi_i(v)$ is defined as follows in [2]:

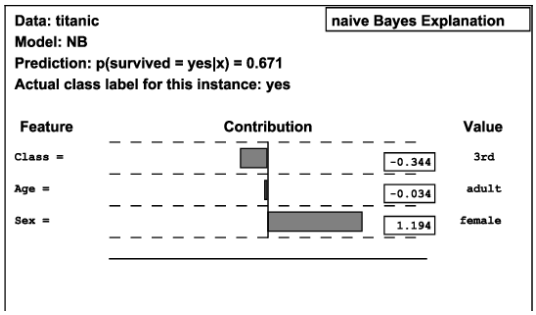

Figure 1: Feature importance attributions computed with Shapley values for the decision of a naive Bayes model on an instance of the Titanic dataset. From [1].

Definition 1.1 *Shapley value*

$$\varphi_i(v) = \frac{1}{n} \sum_{\mathcal{S} \subset \mathcal{F} \setminus \{i\}} \binom{n-1}{|\mathcal{S}|}^{-1} \big(v(\mathcal{S} \cup \{i\}) - v(\mathcal{S})\big) \tag{1}$$

Therefore, the Shapley value for a player i is computed by going through all the coalitions of players not containing i and looking at the payoff difference with and without the player i (with a normalization factor).

The Shapley value of a coalitional game $(\mathcal{F}, v)$ is the only solution that respects the four following powerful axioms, as described in [3]:

- *Efficiency*: The worth of the coalition containing all players is distributed losslessly among the players :
$$\sum_{i \in \mathcal{F}} \varphi_i(v) = v(\mathcal{F}) - v(\emptyset) \tag{2}$$
- *Null Player*: If a player contributes nothing to any coalition, then its Shapley value is zero :
$$\forall i \in \mathcal{F} \quad \big[\forall \mathcal{S} \subset \mathcal{F} \setminus \{i\} \quad v(\mathcal{S} \cup \{i\}) = v(\mathcal{S})\big] \Longrightarrow \varphi_i(v) = 0 \tag{3}$$
- *Symmetry*: Any two players that play equal roles amongst all coalitions receive the same Shapley value :
$$\forall i, j \in \mathcal{F} \quad \big[\forall \mathcal{S} \subset \mathcal{F} \setminus \{i, j\} \quad v(\mathcal{S} \cup \{i\}) = v(\mathcal{S} \cup \{j\})\big] \Longrightarrow \varphi_i(v) = \varphi_j(v) \tag{4}$$
- *Additivity*: Let v_1 and v_2 be two characteristic funtions, the Shapley value respects the additivity of functions :
$$\varphi(v_1 + v_2) = \varphi(v_1) + \varphi(v_2) \tag{5}$$

1.2 Shapley values for feature explanation

In ML, we treat each feature as a player, and the Shapley values allocate the predicted value of a specific instance fairly amongst all the features. Thus, the Shapley values represent the contribution (positive or negative) of each feature in the decision to explain.

Formally, let $x \in \mathbb{F}$ be the instance to explain where $\mathbb{F} = \mathbb{F}_1 \times \cdots \times \mathbb{F}_2$ is the input space of our model (each $\mathbb{F}_i$ is the space of the feature i), and $f(x)$ is the predicted value of our model for a specific class. We consider the coalitional game represented by $\mathcal{F} = \{1, \ldots, n\}$ the set of numbered features that will be our players, and the characteristic function $v : \mathcal{P}(\mathcal{F}) \longrightarrow \mathbb{R}$ defined as follows:

$$\forall \mathcal{S} \subset \mathcal{F} \quad v(\mathcal{S}) = \mathbb{E}[f(X) | X_{\mathcal{S}} = x_{\mathcal{S}}] \tag{6}$$

where $X_{\mathcal{S}} = x_{\mathcal{S}}$ means that instances X and x share the same values for the features in $\mathcal{S}$. The characteristic function is the mean value of the predictions on inputs that share the same values for the features in $\mathcal{S}$.

Finally, we can define the Shapley value of a feature i in the wish of explaining the prediction of an input x as follows:

Definition 1.2 *Shapley value for a feature i*

$$\varphi_i = \frac{1}{n} \sum_{\mathcal{S} \in \mathcal{F} \setminus \{i\}} \binom{n-1}{|\mathcal{S}|}^{-1} \left(\mathbb{E}[f(X)|X_{\mathcal{S}\setminus\{i\}} = x_{\mathcal{S}\setminus\{i\}}] - \mathbb{E}[f(X)|X_{\mathcal{S}} = x_{\mathcal{S}}] \right) \tag{7}$$

Remark. Every Shapley value computed will be based on the choice of the distribution of the reference instances X, and we will see later how it matters. Moreover, we can deduce from the above definition 1.2 and the efficiency axiom 2 the following property:

$$\sum_{i \in \mathcal{F}} \varphi_i(v) = \mathbb{E}[f(X)|X_{\mathcal{F}} = x_{\mathcal{F}}] - \mathbb{E}[f(X)|X_{\emptyset} = x_{\emptyset}] = f(x) - \mathbb{E}[f(X)] \tag{8}$$

Therefore, the sum of the Shapley values of all features equals to the predicted value minus a *reference contribution* that is the mean of all predictions (for inputs based on a specific distribution).

1.3 Approximation

As you may have noticed, computing exact Shapley values has an exponential complexity in terms of the number of features. Indeed, it requires exploring all the subcoalitions $\mathcal{S}$ of $\mathcal{F}\setminus\{i\}$. In practice, this problem becomes intractable when playing with hundreds of features [1]. Moreover, the calculation of the characteristic function $v(\mathcal{S})$ is also intractable as it requires exploring the input space $\mathbb{F}$.

Both issues are solved notably in [2] by using sampling methods. For the former, subcoalitions $\mathcal{S} \subset \mathcal{F}\setminus\{i\}$ are sampled by taking all the features preceding i in random orderings O_j of $\mathcal{F}$: we define $\mathcal{S}_j = Pre_i(O_j)$. For the latter, ehe reference instances X are sampled following the chosen distribution.

Remark. These two approximation solutions allow us to build confidence intervals over the computed Shapley values. Each feature contribution can now be represented as an interval. This is particularly interesting because this gives us more information whether an approximation of the contribution is relevant or not. If a Shapley value is approximated to be close to zero, its confidence interval may take values from negatives to positives values, meaning that we cannot deduce any valuable information about whether this feature contributes positively or negatively in the decision. Remember that if we could compute the exact Shapley value, this case would not exist.

1.4 Different methods

Several explanation methods based on Shapley values have been proposed through the past years. What is described above is the original SHAP (SHapley Additive exPlanations) framework, that serves as the groundwork for newer methods. Various papers pinpointed the shortcomings of SHAP ([4][3]), and mainly the fact that it can attribute more importance to features having less influence (or even no influence at all) in the decision.

On the one hand, [4] claims that this issue can be solely attributed to the characteristic function used, and the authors propose new characteristic functions based on abductive and contrastive explanations (AXps and CXps). They introduce a similarity predicate σ defined below and one example of characteristic function v_a associated :

$$\sigma(y) = \begin{cases} 1 & \text{if } f(y) \approx f(x) \\ 0 & \text{else} \end{cases} \qquad v_a(\mathcal{S}) = \begin{cases} 1 & \text{if } \mathbb{E}[\sigma(X)|X_{\mathcal{S}} = x_{\mathcal{S}}] = 1 \\ 0 & \text{else} \end{cases} \tag{9}$$

Concretely, $v_a(\mathcal{S}) = 1$ if all the instances sharing the same values as x on the features in $\mathcal{S}$ get predicted a similar value (same class in the case of classification). Therefore, $v_a(\mathcal{S})$ is equivalent as knowing if $\mathcal{S}$ is a weak abductive explanation (wAXp) of x. v_a and the other proposed characteristic functions respect two important properties: *strong value independance* (mapping the input space into another one by a surjection doesn't change the value of the characteristic function) and *feature irrelevancy* (a feature that doesn't influence the prediction gets attributed a Shapley value of 0). These properties are desirable to get around the previously mentionned issues.

On the other hand, [2] pinpoint the importance of the choice of the reference distribution, noted $\mathcal{D}^{ref}$, used in the characteristic function to compute $\mathbb{E}[f(X)|X_{\mathcal{S}} = x_{\mathcal{S}}]$. Indeed, we could decide that X follows the input distribution (distribution of the dataset) $\mathcal{D}^{inp}$ (used in SHAP and KernelSHAP), the joint-marginal distribution of features $\mathcal{D}^{J.M.}$ (used in QII), or even a uniform distribution over the input space $\mathcal{D}^{unif}$. While chosing $\mathcal{D}^{inp}$ takes into account only instances present in the dataset and can suffer from the proeminance of some feature values, chosing $\mathcal{D}^{J.M.}$ assumes independance between all the features, and like $\mathcal{D}^{unif}$, can sample unrealistic instances that will have an impact in the computed attributions. These choices are directly reflected in the computed SHAP scores, revealing why different methods can give very different SHAP scores. Therefore, the paper proposes the Formulate-Approximate-Explain framework when facing this choice, saying that we should firstly formulate a contrastive question related to the classification task to pin down the desired reference distribution. The choice in distribution lies in the question of "Why does this instance received this score *in contrast of* these other references".

Finally, there exists model-specific SHAP-based methods for explaining decisions of classifiers with known designs. For instance, [5] presents Eject, intended to give more model-related explanations while solving shortcomings of previous TreeSHAP method that computes SHAP scores on Tree-based models (classification trees and trees ensembles models like random forests). Eject computes the utility of a coalition of features for a specific instance by going down the decision branch and *ejecting* (returning the predicted value of the previously visited node) when the branch uses a feature not present in the coalition. This notably ensures the *local dummy player* property: a feature not used in the prediction gets a score of 0. Also, Eject method allows to compute exact Shapley values rather than approximations. On the other hand, CNN SHAP proposed in [6] aims to compute SHAP scores on convolutive neural networks (more precisely CNN text classification models), and thus on non-structured data. CNN SHAP firstly let the convolutive layers of the model extract the semantic features, and compute the SHAP scores on these extracted features, while keeping in memory the part of the input corresponding to each feature. Then, there's a deduplication step for features having similar semantic meanings and a merging step for features whose corresponding input parts overlap. Finally, the computed SHAP scores can directly be attributed to the input areas.

1.5 Synthesis

In summary, Shapley-based methods give local explanation of feature contributions. While some model-specific methods ensure the tractability of exact computation of Shapley values, this is an exponential complexity problem in terms of the number of feature subsets, solved by sampling-based approximation methods. Different methods have been developped to surpass the shortcomings of the original SHAP framework, by modifying the characteristic function used in the computation of the Shapley values, or the distribution of sampled reference instances that plays a major role in the liability of the contribution scores.

2 Perturbation model-agnostic methods

In this section, we present a set of perturbation method used in black-box models explanation. A perturbation method can be defined by the generation of perturbed samples (ex: noised features in model representation or perturbed input tensor) at the input of the black-box model. The objective is to compare the model behavior (the prediction) between a set

of different perturbed inputs and the original one. Then, we can determine which one is more important for the prediction. It allows the explanation model to detail why the model took a specific decision and why. The following methods register in this type. The first one, LIME (Local Interpretable Model-Agnostic Explanation) focus on a linear approximation of local model behavior. Secondly, the Anchors method provides an amelioration of LIME by introducing the concept of IF-THEN rules to provide a better user-experience. Finally, we will briefly talk about add-ons of those methods in specific cases.

Remark (Notation). Let's introduce some notations to identify which objects we will talk about. We denote with f a black-box model we want to explain, then $f : X \longrightarrow Y$ where X is the set of instances $x \in X$ and Y the different possible predictions $f(x)$. In this section, we will talk about linear explanation model, denoted g. To identify which input representation we use, we denote by x' the interpretable representation of an input x in the black-box model f. Then, if we construct a set of perturbed interpreted samples $\mathcal{Z}$ from x', this set contains perturbed version of x' called z'. Finally, $\Omega(g)$ express the complexity of the model g, generally defined by the number of non-zeros weights in g.

2.1 LIME (Local Interpretable Model-Agnostic Explainer)

LIME founded the principe of perturbation method for classifiers explanation [7]. It provides a local explanation of a black-box model behavior $f : \mathbb{R}^d \longrightarrow \mathbb{R}$ by training a simple explanation model g over perturbed samples. This method is *model-agnostic* because it doesn't depend on the black-box model architecture. f is considered as an oracle used to train g.

Formally, for an original interpretable representation $x' \in \{0,1\}^{d'}$ of a sample $x \in \mathbb{R}^d$, it generates a set of perturbed samples z' which are drawn by selecting uniformly some x' non-zero components. This random sampling is weighted by π_x which defines a locality around x' denoted $\mathcal{Z}$. In practice, π_x is an exponential kernel applied to a distance D. To find the optimal model g, we can define a loss function $\mathcal{L}(f, g, \pi_x)$ telling how unfaithful g is in approximating f on $\mathcal{Z}$. The objective is then to minimize $\mathcal{L}$ and having $\Omega(g)$ low enough for human comprehension. The LIME explanation is then produced by:

$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \qquad (10)$$

with G the set of available explanation models [7]. Using $\mathcal{L}$ and $\Omega(g)$ to find the best model g ensures *interpretability* and *local fidelity* on the black-box model explanation.

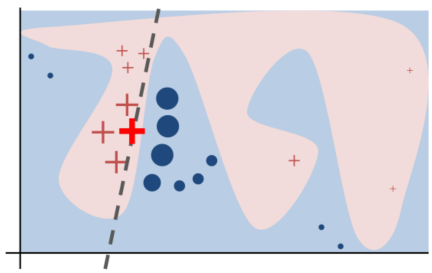

Figure 2: **LIME linear local approximation. Black-box model f classes is represented in blue/gray. The red cross represents an explained instance and the others symbols are the perturbation samples generated from the original. The dotted line represents the local approximation of g.**

Finally, the local explanation of the model is provided by the weights w_g trained with $g(z') = w_g \cdot z'$.

As stated at beginning, LIME provides a *local* explanation of f. To ensure a global explanation, the paper proposes a Sub-modular Picks LIME (SP-LIME) which selects a set local and non-redundant LIME explanations to be given to the user [7]. To choose those explanations, the paper introduces the concept of coverage of a set of instances which defines the importance of those instances.

Definition 2.1 *Coverage*
Let g be an explanation model and $g_i = \xi(x_i)$ the explanation of the instance x_i,

we construct the $|X| \times d'$ explanation matrix $\mathcal{W}$ with $\mathcal{W}_{i,j} = |w_{g_{i,j}}|$. Then, I_j is the j-column of W and denotes the global importance of that component in the explanation space. Then, the coverage of a set of instances V is defined as the total importance of the features that appear at least one instance in V:

$$c(V, \mathcal{W}, I) := \sum_{j=1}^{d'} \mathbb{1}_{\{\exists i \in V, \mathcal{W}_{i,j} > 0\}} I_j \tag{11}$$

Finally, from a maximum number of global explanations B set by the user, the pick problem is defined by:

$$Pick(\mathcal{W}, I) = \underset{V, |V| \leqslant B}{\arg\max}\ c(V, \mathcal{W}, I) \tag{12}$$

We want to select the highest coverage and non-redundant explanations to provide a global one.

Unfortunately, this system has limits. First, in practice, the pick problem is intractable, so a greedy algorithm is used to compute it. Secondly, LIME algorithm doesn't define clear boundaries of the explanation instances. Indeed, all black-box models cannot be locally approximated by a linear one [8]. Finally, the explanation results depend on the LIME parameters such as B, the distance metric D and the kernel π_x. Changing one of them can change the interpretation and undermining the reliability of this method [9].

2.2 Anchors

To address the lack of clear coverage boundaries on LIME, Ribeiro, Singh, and Guestrin [8] introduced anchor-LIME (aLIME). This LIME evolution uses IF-THEN rules to 'anchor' a prediction. This transition to linear importance to rule-based invariance laid the foundation of the Anchors method. Thus, they are defined by short and disjoint rules which are preferred by users for explanation [10]. This method can be applied on different set of domains (tabular, text, images, etc...) for different tasks (classification, prediction, text generation, etc...).

Definition 2.2 *Anchors*

Formally, Anchors are built on a set of predicates $\{a_i\}$ called a "rules". We can also see it as a function of instances $x \in X$:

$$A : \begin{cases} X \longrightarrow \{0, 1\} \\ x \longmapsto \begin{cases} 1 \text{ if all rules are true for } x \\ 0 \text{ otherwise} \end{cases} \end{cases}$$

to link a set of rules A on a prediction $f(x)$ (where $f : X \longrightarrow Y$ is the black-box model). Finally, A is an Anchor of x if $A(x) = 1$ and A is a sufficient condition for the prediction $y = f(x)$:

$$\mathbb{E}_{\mathcal{D}(z|A)}[1_{f(x)=f(z)}] \geqslant \tau \tag{13}$$

with τ the desired level of precision of the Anchor on x [10]. Here, $\mathcal{D}(x \mid A)$ represents the distribution of neighbors of x which satisfy the Anchor A.

Computing Anchors is the difficult part of the process. This is a combinatorial optimization problem if we decide to choose anchors which satisfy a precision constraint with high probability:

$$\mathbb{P}(prec(A) \geqslant \tau) \geqslant 1 - \delta$$

with $prec(A) = \mathbb{E}_{\mathcal{D}(z|A)}(1_{f(x)=f(z)})$ called the Anchors "precision". Among those anchors, we choose the largest coverage ones, where coverage is :

$$cov(A) = \mathbb{E}_{\mathcal{D}(z)}(A(z)) \tag{14}$$

Finally, the optimization problem can be described by :

$$\max_{\mathbb{P}(prec(A)\geqslant\tau)\geqslant 1-\delta} cov(A) \tag{15}$$

Because the number of all possible Anchor is exponential, we can perform an exhaustive search. That's why the article introduced the Bottom-Up approach:

1. *Initialization*: The anchor A is initialized as an empty rule (applying to all instances).
2. *Candidate Generation*: In each iteration, a set of candidate rules $\mathcal{A}$ is generated by extending A with one additional feature predicate $\{a_i\}$: $\mathcal{A} = \{A \wedge a_i, A \wedge a_{i+1}, A \wedge a_{i+2}, ...\}$
3. *Selection via Multi-Armed Bandit (MAB)*: To identify the best candidate without an exhaustive number of calls to f, the problem is formulated as a pure-exploration multi-armed bandit problem. The *KL-LUCB* algorithm is used to find the rule with the highest precision using a minimal set of samples [10].
4. *Termination*: The process terminates when a rule meets the statistical precision constraint: $\mathbb{P}(prec(A) \geqslant \tau) \geqslant 1 - \delta$.

To address the limitations of a purely greedy search, the algorithm extends the construction to a *Beam Search* of size B. It maintains a set of B current candidates instead of a single one. Among the multiple anchors identified that satisfy the precision threshold τ, the algorithm outputs the one with the highest coverage $cov(A)$, effectively optimizing the original problem.

To provide a global understanding of the model, the article adapts a *Submodular Pick* (SP) procedure. The goal is to select a set of K anchors that are non-redundant and cover as many instances in the validation set similar to the approach used in LIME [7, 10].

2.3 Related

Let's talk about related topics about Anchors and LIME and specifically about improvements provided in specific cases.

The first one is an amelioration of LIME weights for an instance. The related work is motivated by the fact that LIME system define the importance of a feature only by the distance between its original representation and the noised one in the model representation. With ILIME, the paper proposes a new method which consider the black-box loss function value between the two instances, this defines the *influence* of a feature. Formally, it is defined as scalar product between original weight of the prediction on the perturbed feature and the loss function gradient evaluated at this point. The final "importance" value of a feature on the model's prediction is defined as the product of its influence and its original LIME weight. Thus, a synthetic instance will have an important weight if its representation is near to the original feature one, and if it is able to modify the model prediction. To provide a global explanation of the model behavior, ILIME uses a set of local explanations and defines a Dendogram with hierarchical clustering on explanations (weights). It differs from LIME where the user needed to define the number of global explanation we wanted.

Secondly, Ribeiro, Singh, and Guestrin [8] presents the evolution of LIME system by adding Anchors to clearly define LIME coverage boundaries. Indeed, by construction LIME perturbs the output representation x' of a black-box model f by selecting random x' components according to a distribution π_x. However, this π_x doesn't define a clear zone around x' where the system select the perturbed inputs to give to the models, ones can be very close and the other very far. So that human cannot really understand which feature is more important than another. That's why, aLIME is an IF-THEN rules-based system that "anchors" a prediction features such that changing the others don't change de black-box model output. By this approach, precision (defined as how accurate humans are in those predictions) is needed but a too large set of rules only specific to a single instance is not useful for model understandability. Then the combinatorial aLIME problem if to find the shortest Anchor with high precision.

As presented in the previous section, SHAP-Scores in an intractable problem. Fortunately, Arenas et al. [11] presents an efficient algorithm to compute SHAP-scores within specific classification models. The SHAP-score is a prominent explanation method for classifiers that assigns a numerical value to each input feature based on its contribution to the decision process. In general, computing SHAP-scores is computationally intractable due to the model's complexity, often being #P-hard. However, this paper focuses on *Boolean Classifiers*, which return either 1 (accept) or 0 (reject) for a given input. These models are represented as *deterministic* and *decomposable* Boolean circuits, a class that encompasses models like Bayesian Classifiers, Random Forests, and Decision Trees. In this framework, the authors provide a polynomial-time algorithm to compute SHAP-scores over tractable Boolean circuits, initially assuming a uniform distribution over inputs. Both the determinism and decomposability properties are strictly necessary for this tractability:

- Determinism ensures that for every OR gate, no two inputs can be true at the same time for any given assignment.
- Decomposability requires that for every AND gate, the sets of features in its subcircuits are pairwise disjoint (they share no variables).

The core contribution of the paper is the reduction of the SHAP-score computation to a #SAT problem (model counting). The authors then define a polynomial-time algorithm to solve this specific #SAT problem using a *bottom-up induction* process.

Finally, we can discuss an implementation of SHAP-Scores and Anchors in financial fraud detection. Indeed, Thanathamathee et al. [12] presents a real-case application of SHAP-values and Anchors in fraud detection systems. While traditional statistical methods are common, they struggle with the complex nature of modern fraud. The authors enhance the XGBoost model by using a novel SHAP-instance weighting method to prioritize learning from the most informative samples. To solve the massive class imbalance in the Thailand Stock Exchange dataset, they combine RENN and SMOTE techniques before optimizing the model with Optuna and Hyperband. The resulting model effectively differentiates between non-fraudulent, fraudulent, and "grey area" behaviors. By reaching a perfect recall (1.000) for fraudulent cases, this approach sets a new standard for accuracy and trust. Furthermore, Anchors are used to translate "black-box" predictions into easy-to-understand IF-THEN rules, providing clear insights for risk management. However, integrating these SHAP and Anchor systems increases the computational load and total processing time.

In summary, LIME defined the principle of perturbation explainer by learning weights which denotes an instance importance in the black-box decision process. Then, Anchors enhanced this model by adding IF-THEN rules which allows to give clear instances coverage to the user. Nevertheless, this precision is difficult to reach because of the exponential complexity fo this type of computation. In practice, Anchors and LIME are used in specific domains where explainability is necessary to trust the model.

3 Comparison of all methods

In this section, we give a comparison table of all the previously mentioned explanation methods.

4 Conclusion

References

[1] Erik Strumbelj and Igor Kononenko. "An Efficient Explanation of Individual Classifications using Game Theory". In: *J. Mach. Learn. Res.* 11 (Mar. 2010), pp. 1–18. ISSN: 1532-4435. URL: https://dl.acm.org/doi/10.5555/1756006.1756007 (visited on 02/21/2026).

Table 1: Comparative analysis of explanation methods.

Method	Agnostic	Output Type	Key Features / Limitations
LIME	Yes	Weights	Local linear surrogate. Fast but lacks clear coverage boundaries and can be unstable.
Anchors	Yes	IF-THEN Rules	Provides sufficient conditions for prediction. High precision but exponential search space.
ILIME	Yes	Weights	Enhances LIME with influence-based weighting and hierarchical clustering for global views.
aLIME	Yes	Rules	Hybrid approach using rules to stabilize LIME's local neighborhood.
SHAP-Logic	No	Scores	Focuses on Boolean circuits tractability. Polynomial-time computation for specific models.

[2] Luke Merrick and Ankur Taly. "The Explanation Game: Explaining Machine Learning Models Using Shapley Values". en. In: *Machine Learning and Knowledge Extraction*. Ed. by Andreas Holzinger et al. Cham: Springer International Publishing, 2020, pp. 17–38. ISBN: 978-3-030-57321-8. DOI: 10.1007/978-3-030-57321-8_2.

[3] Daniel Fryer, Inga Strümke, and Hien Nguyen. "Shapley Values for Feature Selection: The Good, the Bad, and the Axioms". In: *IEEE Access* 9 (2021), pp. 144352–144360. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3119110. URL: https://ieeexplore.ieee.org/abstract/document/9565902 (visited on 02/16/2026).

[4] Olivier Létoffé, Xuanxiang Huang, and Joao Marques-Silva. "Towards trustable SHAP scores". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 17. 2025, pp. 18198–18208. URL: https://ojs.aaai.org/index.php/AAAI/article/view/34002 (visited on 02/05/2026).

[5] Thomas W. Campbell et al. "Exact Shapley values for local and model-true explanations of decision tree ensembles". In: *Machine Learning with Applications* 9 (Sept. 2022), p. 100345. ISSN: 2666-8270. DOI: 10.1016/j.mlwa.2022.100345. URL: https://www.sciencedirect.com/science/article/pii/S2666827022000500 (visited on 02/21/2026).

[6] Wei Zhao et al. *SHAP values for Explaining CNN-based Text Classification Models*. July 2021. DOI: 10.48550/arXiv.2008.11825. URL: http://arxiv.org/abs/2008.11825 (visited on 02/21/2026).

[7] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. ""Why Should I Trust You?": Explaining the Predictions of Any Classifier". en. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. San Francisco California USA: ACM, Aug. 2016, pp. 1135–1144. ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939778. URL: https://dl.acm.org/doi/10.1145/2939672.2939778 (visited on 02/05/2026).

[8] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. *Nothing Else Matters: Model-Agnostic Explanations By Identifying Prediction Invariance*. Nov. 2016. DOI: 10.48550/arXiv.1611.05817. URL: http://arxiv.org/abs/1611.05817 (visited on 02/21/2026).

[9] Rahel Wilking, Matthias Jakobs, and Katharina Morik. "Fooling Perturbation-Based Explainability Methods". In: *Workshop on Trustworthy Artificial Intelligence as a part of the ECML/PKDD 22 program*. Grenoble, France, France: IRT SystemX [IRT SystemX], Sept. 2022. URL: https://hal.science/hal-03773429 (visited on 02/21/2026).

[10] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Anchors: High-precision model-agnostic explanations". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018. URL: https://ojs.aaai.org/index.php/aaai/article/view/11491 (visited on 02/05/2026).

[11] Marcelo Arenas et al. "The Tractability of SHAP-Score-Based Explanations for Classification over Deterministic and Decomposable Boolean Circuits". en. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 35.8 (May 2021), pp. 6670–6678. ISSN: 2374-3468. DOI: 10.1609/aaai.v35i8.16825. URL: https://ojs.aaai.org/index.php/AAAI/article/view/16825 (visited on 02/21/2026).

[12] Putthiporn Thanathamathee et al. "SHAP-Instance Weighted and Anchor Explainable AI: Enhancing XGBoost for Financial Fraud Detection". In: *Emerging Science Journal* 8 (Dec. 2024), pp. 2404–2430. DOI: 10.28991/ESJ-2024-08-06-016.